

BugVault Technical Specification

End-to-end production documentation for **BugVault** — a full-stack personal bug tracking, root-cause logging, and solution reference web application built with Node.js, Express, SQLite, and modern web standards.

Author: Venkata Ajay Kumar
GitHub: @venkataajaykumar19
Repository: venkataajaykumar19/BugVault
Date: September 16, 2026
Verification: 39/39 Local Tests Passing

SECTION 1 System Architecture

BugVault is architected as a lightweight, decoupled full-stack client-server system designed to eliminate repetitive debugging by tracking programming errors, their root causes, working fixes, and category classifications. The application adheres to strict minimalism, zero bloated frameworks, and robust relational persistence.

1. WHERE IS THE FRONTEND DEPLOYED?

GitHub Pages (Static Web Hosting)

Served via HTTPS from `https://venkataajaykumar19.github.io/BugVault/frontend/index.html`

2. WHERE IS THE BACKEND DEPLOYED?

Render (Cloud Web Service)

Hosted at `https://bugvault-api.onrender.com` running Node.js runtime environment.

3. WHAT DATABASE IS USED?

SQLite 3 (Relational Embedded Engine)

Driven via native `better-sqlite3` (v11.8.1) with `PRAGMA foreign_keys = ON` & Write-Ahead Logging (WAL).

4. WHERE IS THE DATABASE HOSTED?

Co-located on Server Filesystem

Stored locally in `backend/database.sqlite` on the backend host with atomic ACID transactions.

5 & 6. DOES BACKEND CALL EXTERNAL APIS? WHICH EXTERNAL APIS?

No External APIs Called (100% Self-Contained)

The backend performs zero outbound HTTP requests. It relies entirely on internal business logic, schema validation, and local SQLite persistence, ensuring zero third-party latency, zero rate-limiting vulnerabilities, and sub-millisecond query execution.

— SYSTEM COMPONENT & DATA FLOW DIAGRAM —

User interacts via Web Browser (Desktop / Mobile)

FRONTEND CLIENT (UI)

HTML5 Semantic Structure • Responsive CSS3 • Vanilla ES6+ JavaScript

DEPLOYED ON: GITHUB PAGES

URL: `https://venkataajaykumar19.github.io/BugVault/frontend/index.html`

HTTPS / REST JSON API Requests (CORS Enabled)

EXPRESS REST API SERVER (BACKEND)

Node.js (v18+) • Express 4.21.2 • cors • dotenv • Prepared Statements

DEPLOYED ON: RENDER WEB SERVICE

URL: `https://bugvault-api.onrender.com` (Port 3000)

Synchronous In-Process C++ Bindings (better-sqlite3) • PRAGMA foreign_keys = ON

SQLITE 3 RELATIONAL DATABASE (2 ENTITIES)

Table: **categories** (id, name) • Table: **bugs** (id, title, error, cause, solution, date, category_id)
Relationship: **categories 1** — **N bugs** (Foreign Key: bugs.category_id → categories.id)

STORAGE: BACKEND/DATABASE.SQLITE (WAL MODE)

Step-by-Step Architecture Flow (End-to-End User Action Lifecycle)

The complete path traversed by data from user interaction to disk storage and back to visual feedback follows a deterministic, 10-step lifecycle:

- 1 User Interaction & Category Selection:** The user opens the web client in their browser, selects an error category from `#bug-category` (loaded dynamically via `GET /categories`), and fills in the bug entry form in `frontend/index.html` (inputs: `#bug-title`, `#bug-error`, `#bug-cause`, `#bug-solution`, `#bug-date`), then clicks `#submit-btn`.
- 2 Client Event Capture:** In `frontend/script.js`, the submit listener on `#bug-form` intercepts the form submission via `e.preventDefault()` to prevent native page refresh.
- 3 Client Validation & Serialization:** `frontend/script.js` extracts trimmed values, verifies that no field is blank and that a valid `category_id` integer is selected. It activates the loading spinner, disables the submit button, and serializes the payload into JSON: `{ title, error, cause, solution, date, category_id }`.
- 4 HTTP Request Dispatch:** The client issues an asynchronous `fetch()` request targeting `${BACKEND_URL}/bugs` with HTTP method `POST`, header `Content-Type: application/json`, and the stringified JSON payload.
- 5 Network Ingestion & Middleware:** The Render proxy routes the HTTPS request to the Node.js Express process. Express in `backend/server.js` invokes `cors()` (validating cross-origin headers) followed by `express.json()` (parsing JSON into `req.body`).
- 6 Controller & Foreign Key Validation:** The route handler `app.post('/bugs', ...)` enforces that all required fields exist and queries the `categories` table to ensure `category_id` refers to an existing entity. If invalid or missing, it aborts with HTTP 400 immediately.
- 7 Database Execution:** The handler executes the pre-compiled SQLite statement `insertBugStmt.run(...)` via `better-sqlite3`, inserting the record into table `bugs` in `backend/database.sqlite` with `category_id` and writing to the WAL journal.
- 8 Record Retrieval & SQL JOIN:** The handler fetches the newly created record via `selectBugByIdStmt.get(info.lastInsertRowid)`, which executes a `LEFT JOIN categories ON bugs.category_id = categories.id`, returning the complete bug object with `category_name` with HTTP status `201 Created`.
- 9 Client Ingestion & Notification:** The client promise resolves. `frontend/script.js` displays a green alert banner (`#alert-banner`) reading "Bug record #{id} saved successfully!", resets form fields, and restores the category dropdown.
- 10 DOM Reconciliation:** The frontend invokes `fetchBugs()`, which calls `GET /bugs`. The returned JSON array is rendered by `renderBugsList()` into styled bug cards inside `#bugs-container` with category tags (e.g., `JavaScript`), updating the badge count.

SECTION 2 API Documentation

The BugVault backend exposes a strictly scoped REST API implemented in `backend/server.js`. The backend registers exactly **6 HTTP routes**: 1 operational health check endpoint, 1 relational category reading endpoint, and 4 core CRUD application endpoints for the primary `bugs` resource.

OPERATIONAL / HEALTH

GET /health

Server uptime & liveness check

Purpose: Verifies that the Express runtime process is active and responsive. Queried on frontend load by `checkBackendHealth()` in `frontend/script.js` to update the UI status badge (`#backend-status-badge`).

Request Headers: `Accept: application/json` • **Request Body:** None

Status Code	Meaning	Response Payload
200 OK	Service is healthy and ready to accept traffic.	<code>{ "status": "ok" }</code>
500 Server Error	Service encountered an unhandled process failure.	<code>{ "error": "Internal Server Error", "message": "... " }</code>

RELATIONAL ENTITY **GET** **/categories***List all available categories*

Purpose: Retrieves all category entities from SQLite table `categories`. Populates the frontend category selection dropdown.

Request Headers: `Accept: application/json` • **Request Body:** None

Status Code	Condition	Response Format
200 OK	Categories retrieved	Array of category objects: [{ "id": 1, "name": "JavaScript" }, { "id": 2, "name": "React" }, ...]
500 Server Error	Database query failure	{ "error": "Failed to fetch categories", "message": "A database error occurred." }

APPLICATION CRUD **POST** **/bugs***Create bug record (CREATE)*

Purpose: Validates payload fields, verifies foreign key existence in `categories`, and inserts a new bug record into table `bugs`.

Request Headers: `Content-Type: application/json`, `Accept: application/json`

Field	Type	Required	Description
title	string	Yes	Concise summary of the bug/error (non-empty)
category_id	integer	Yes	Valid foreign key referencing <code>categories.id</code>
error	string	Yes	Raw error message, stack trace snippet, or symptom
cause	string	Yes	Root cause explaining why the error occurred
solution	string	Yes	Step-by-step resolution or code modifications applied
date	string	Yes	Date when encountered in ISO format (YYYY-MM-DD)

```
{
  "title": "Prisma environment error",
  "category_id": 4,
  "error": "Environment variable not found: DATABASE_URL",
  "cause": "DATABASE_URL was missing from .env after fresh git clone",
  "solution": "Added DATABASE_URL to backend/.env and ran npx prisma generate",
  "date": "2026-09-16"
}
```

Status	Condition	Response Format
201 Created	Record created successfully	Returns created bug object with <code>id</code> , <code>category_id</code> , and joined <code>category_name</code> .
400 Bad Request	Missing fields or non-existent <code>category_id</code>	{ "error": "Invalid category", "message": "Category with ID 99 does not exist." }
500 Server Error	Database insertion failure	{ "error": "Failed to create bug record", "message": "A database error occurred." }

Purpose: Queries SQLite for all stored bug records via `LEFT JOIN categories ON bugs.category_id = categories.id`, ordered by `bugs.id DESC`.

Request Headers: Accept: application/json • **Request Body:** None

Status	Condition	Response Format
200 OK	Records retrieved	Array: [{ "id": 1, "title": "...", "category_id": 4, "category_name": "Database", "error": "...", "cause": "...", "solution": "...", "date": "..." }]
500 Server Error	Database query failure	{ "error": "Failed to fetch bug records", "message": "A database error occurred." }

Purpose: Updates all fields of an existing bug, including updating or reassigning its foreign key `category_id`.

URL Parameters: `id` (Integer, required): The unique record ID.

Headers: Content-Type: application/json, Accept: application/json

```
{
  "title": "Prisma environment error (Resolved)",
  "category_id": 4,
  "error": "Environment variable not found: DATABASE_URL",
  "cause": "DATABASE_URL was missing from .env after fresh git clone",
  "solution": "Added DATABASE_URL to backend/.env and ran npx prisma generate",
  "date": "2026-09-16"
}
```

Status	Condition	Response Format
200 OK	Update successful	Returns complete updated bug object with joined <code>category_name</code> .
400 Bad Request	Non-integer ID, missing fields, or invalid <code>category_id</code>	{ "error": "Invalid category", "message": "Category with ID 99 does not exist." }
404 Not Found	Bug ID does not exist	{ "error": "Bug record not found", "message": "No bug found with ID 99." }
500 Server Error	Database update failure	{ "error": "Failed to update bug record", "message": "A database error occurred." }

Purpose: Removes the specified bug record from SQLite. In compliance with the One-to-Many architecture, deleting a bug does NOT delete its parent category (no cascading deletion).

URL Parameters: `id` (Integer, required): The unique record ID • **Headers:** Accept: application/json

Status	Condition	Response Format
200 OK	Deletion successful	{ "success": true, "message": "Bug with ID 1 successfully deleted.", "deletedId": 1 }
400 Bad Request	Non-integer ID parameter	{ "error": "Invalid bug ID", "message": "Bug ID must be a positive integer." }
404 Not Found	No record found with specified ID	{ "error": "Bug record not found", "message": "No bug found with ID 99." }
500 Server Error	Database deletion failure	{ "error": "Failed to delete bug record", "message": "A database error occurred." }

Global Catch-All Error Handling Middleware:

In `backend/server.js`, an Express error-handling middleware intercepts malformed JSON syntax errors in request bodies and returns an immediate HTTP 400 Bad Request: `{ "error": "Malformed JSON", "message": "The request payload contains invalid JSON syntax." }`.

SECTION 3 Database Design

BugVault uses **SQLite 3** driven by `better-sqlite3` (v11.8.1). SQLite was chosen intentionally over heavy client-server engines because it provides zero configuration overhead, instantaneous sub-millisecond execution, complete file portability, and ACID transaction guarantees.

Property	Setting	Technical Justification
Database Engine	SQLite 3	Embedded, serverless SQL database stored co-located with the Node.js runtime process.
Driver / Library	<code>better-sqlite3</code> (v11.8.1)	Synchronous, high-throughput execution with direct V8 memory bindings. Outperforms async drivers for local I/O.
Foreign Keys	<code>PRAGMA foreign_keys = ON;</code>	Enforces referential integrity at the database engine level before any statement executes.
Journal Mode	<code>PRAGMA journal_mode = WAL;</code>	Write-Ahead Logging permits readers to read while writers write, significantly enhancing concurrency.
File Storage	<code>backend/database.sqlite</code>	Single-file database with accompanying <code>-wal</code> (journal) and <code>-shm</code> (shared memory index) files.
Migration Strategy	Automated DDL at Startup	Executed synchronously on server initialization via <code>CREATE TABLE IF NOT EXISTS</code> and safe column checks.

Entity 1: Table `categories`

Stores pre-seeded and user-selectable error classifications:

Column Name	Data Type	Constraints	Default	Description
<code>id</code>	INTEGER	PRIMARY KEY AUTOINCREMENT	Auto-incremented	Unique primary key identifying each category.
<code>name</code>	TEXT	NOT NULL UNIQUE	None	Unique category label (e.g. "JavaScript", "React", "Node.js", "Database", "Git/GitHub").

Entity 2: Table `bugs`

Stores resolved bug reports, root causes, fixes, and foreign key references:

Column Name	Data Type	Constraints	Default	Description
<code>id</code>	INTEGER	PRIMARY KEY AUTOINCREMENT	Auto-incremented	Unique primary key identifying each bug entry.
<code>title</code>	TEXT	NOT NULL	None	Concise headline summarizing the problem.
<code>error</code>	TEXT	NOT NULL	None	Full error message, stack trace snippet, or symptom.
<code>cause</code>	TEXT	NOT NULL	None	Technical explanation of the underlying root cause.
<code>solution</code>	TEXT	NOT NULL	None	Detailed fix, resolution steps, or code patch applied.
<code>date</code>	TEXT	NOT NULL	None	ISO 8601 calendar date string (<code>YYYY-MM-DD</code>).
<code>category_id</code>	INTEGER	REFERENCES categories(id)	NULL	Foreign key linking the bug to its parent category in <code>categories</code> .

Relational Relationship Specification

One-to-Many (1:N) Relationship:

categories 1 ---- N bugs

Cardinality:

One Category can be associated with many Bug records. Each Bug belongs to exactly one Category via bugs.category_id .

Referential Integrity:

Verified through SQLite pragma PRAGMA foreign_key_list(bugs); , which confirms that category_id maps to categories.id with on_delete: NO ACTION (deleting a bug does not delete the category).

Compiled Prepared Statements & Injection Prevention

All database operations use parameterized prepared statements compiled once during server bootstrap in backend/server.js , guaranteeing 100% immunity against SQL injection:

```
-- 1. Insert Statement (Named Parameter Binding with category_id)
INSERT INTO bugs (title, error, cause, solution, date, category_id)
VALUES (@title, @error, @cause, @solution, @date, @category_id);

-- 2. Select All Statement (Relational SQL LEFT JOIN)
SELECT b.id, b.title, b.error, b.cause, b.solution, b.date, b.category_id, c.name AS category_name
FROM bugs b
LEFT JOIN categories c ON b.category_id = c.id
ORDER BY b.id DESC;

-- 3. Select by ID Statement (Relational SQL LEFT JOIN)
SELECT b.id, b.title, b.error, b.cause, b.solution, b.date, b.category_id, c.name AS category_name
FROM bugs b
LEFT JOIN categories c ON b.category_id = c.id
WHERE b.id = ?;

-- 4. Update Statement (Named Parameter Binding with category_id)
UPDATE bugs
SET title = @title, error = @error, cause = @cause, solution = @solution, date = @date, category_id = @category_id
WHERE id = @id;

-- 5. Delete Statement (Positional Parameter Binding)
DELETE FROM bugs WHERE id = ?;
```

SECTION 4 Deployment Documentation

BugVault is deployed across two cloud platforms following a decoupled production architecture: GitHub Pages for static frontend assets and Render for the dynamic Node.js backend Web Service.

Deployment Metadata & URLs

Layer	Hosting Platform	Live Production URL	Configuration File
Frontend	GitHub Pages (Static)	https://venkataajaykumar19.github.io/BugVault/frontend/index.html	frontend/index.html
Backend	Render (Web Service)	https://bugvault-api.onrender.com	render.yaml
Health Check	Render Endpoint	https://bugvault-api.onrender.com/health	backend/server.js
Categories API	Render Endpoint	https://bugvault-api.onrender.com/categories	backend/server.js
Repository	GitHub (Public)	https://github.com/venkataajaykumar19/BugVault	Git repository root

Production Storage Clarification (Render Free Tier):

On the Render Free Tier Web Service, the filesystem is **ephemeral**. Any writes to `backend/database.sqlite` persist during an active container lifecycle, but will reset upon container restarts or re-deployments unless a paid Render Persistent Disk is attached. For permanent, guaranteed persistent storage, BugVault is fully verified on local disk storage (where `database.sqlite` persists indefinitely across restarts).

Infrastructure-as-Code (Render Web Service)

Declared in root `render.yaml`:

```
services:
  - type: web
    name: bugvault-api
    env: node
    plan: free
    rootDir: backend
    buildCommand: npm install
    startCommand: node server.js
    envVars:
      - key: PORT
        value: 3000
```

Dynamic Client Environment Switching

In `frontend/script.js`, client dynamically toggles between local backend and Render deployment:

```
const isLocal = window.location.hostname === 'localhost' ||
  window.location.hostname === '127.0.0.1' ||
  window.location.protocol === 'file:';

const BACKEND_URL = isLocal
  ? 'http://localhost:3000'
  : 'https://bugvault-api.onrender.com';
```

Environment Variable Configuration

Variable	Default	Location	Security Rule
PORT	3000	backend/.env • Render Dashboard	Documented in <code>backend/.env.example</code> (tracked). Live <code>.env</code> file is strictly excluded by <code>.gitignore</code> .

Local Reproduction & Execution Guide

```
# 1. Clone repository and install backend dependencies
git clone https://github.com/venkataajaykumar19/BugVault.git
cd BugVault/backend
npm install

# 2. Configure environment and start API server
cp .env.example .env
npm start
# Server listens at http://localhost:3000 (Test: curl http://localhost:3000/health)

# 3. Launch Frontend Client
# Option A: Open directly in Chrome/Edge:
start ../frontend/index.html
# Option B: Run static file server:
npx serve ../frontend

# 4. Execute Automated Verification Suite
node test_crud.js
# Runs all 39 automated assertions covering Health, Categories, POST, GET, PUT, DELETE, and Foreign Keys
```

SECTION 5 Codebase Structure

The BugVault repository is organized into cleanly partitioned client and server directories, enforcing strict separation of concerns between presentation, business logic, relational persistence, and deployment infrastructure.

Annotated Directory Tree (Matches Active Git Repository)

```
BugVault/ ├── .gitignore # Excludes node_modules, .env, *.sqlite*, OS cache | BugVault_Technical_Documentation.pdf # Challenge #10 comprehensive technical documentation | README.md # Project overview, 2-entity schema, routes, live URLs, setup | VIDEO_SCRIPT.md # 3-minute presentation script for video submission | index.html # Root redirect to frontend/index.html | render.yaml # Render Web Service Infrastructure-as-Code descriptor | serve_frontend.js # Utility script to serve frontend files locally | verify_frontend_browser.js # Puppeteer / browser automation test script for categories & CRUD | backend/ # Node.js + Express REST API & SQLite database | .env.example # Environment template tracking PORT=3000 | package-lock.json # Deterministic dependency lockfile | package.json # Dependencies: express, cors, dotenv, better-sqlite3 | server.js # Express server, SQLite categories/bugs setup, 6 routes, error middleware | test_crud.js # Comprehensive automated test suite (39 assertions) | frontend/ # Static web client (GitHub Pages root) | index.html # Accessible HTML5 structure, category selector, semantic form, and cards grid | script.js # Category fetching, CRUD state management, DOM reconciliation, XSS sanitization | style.css # Responsive styling, design tokens, category tags, flexbox/grid layout | [Verification Screenshots] | frontend_verified_initial.png # Browser evidence: initial loaded state with category dropdown | frontend_verified_created.png # Browser evidence: record creation with React category tag | frontend_verified_final.png # Browser evidence: final CRUD & category switch verified state
```

File Responsibilities & Architectural Boundaries

File Path	Layer	Key Responsibilities
backend/server.js	Backend (Core)	Initializes Express, enables CORS, opens SQLite database, enables foreign keys ('PRAGMA foreign_keys = ON'), creates 'categories' & 'bugs' tables, seeds default categories, compiles prepared SQL statements, registers 6 routes, and handles global errors.
backend/test_crud.js	Testing / QA	Autonomous test suite verifying health check, category listing, foreign key constraint enforcement, relational bug creation, SQL JOIN reads, category updates, and preservation after bug deletion (39 assertions).
frontend/index.html	Frontend (Structure)	Semantic HTML5 document featuring header badge, global alert banner, category selector dropdown, two-column split layout (entry form on left, bug cards on right), and loading/empty states.
frontend/script.js	Frontend (Logic)	Fetches categories via 'GET /categories', manages client state ('bugs', 'categories', 'editingBugId'), handles form submission, toggles edit mode with category preselection, dispatches REST API requests, and renders bug cards with category tags.
frontend/style.css	Frontend (Styling)	CSS3 design system using HSL color variables, modern typography, category badges ('.bug-category-tag'), elevation drop shadows, animated loading spinner, and responsive media queries.
render.yaml	DevOps / Release	Declares cloud infrastructure for Render: free tier web service, root directory backend, build command, and startup command.

Engineering Security & Robustness Practices

Vector	Implementation in BugVault	Verification Result
SQL Injection	100% prepared statements with parameterized inputs via better-sqlite3. No string concatenation.	Verified in backend/test_crud.js (SQL strings treated strictly as literal data).
Referential Integrity	'PRAGMA foreign_keys = ON' and explicit controller check verifying 'category_id' exists in 'categories' before insertion/update.	Verified in test_crud.js (invalid 'category_id: 99999' rejected with HTTP 400).
XSS Prevention	All user strings passed through escapeHtml() in frontend/script.js before DOM injection.	Verified (HTML entities &, <, >, ", ' safely escaped).
Secret Protection	.env file gitignored. Only safe template .env.example tracked in git.	Verified via git status and repository file audit (zero secrets committed).
Defensive JSON Parsing	Global Express error handler catches syntax errors from malformed JSON and responds with HTTP 400.	Verified (invalid JSON payloads handled cleanly without process termination or stack trace leaks).